

DOMNIWALLET

Wallet + Payout Service - Especificação Técnica

Sistema Custodial + Payout Multi-Rede: BTC / Liquid / Tron

Documento	Especificação Técnica do Wallet + Payout Service
Versão	1.0.0
Status	Draft v1 (para revisão/aprovação)
Data	Janeiro 2026
Público-Alvo	Desenvolvedores, Arquitetos, Operações
Redes Suportadas	Bitcoin, Liquid Network, Tron
Backend	Go + PostgreSQL + Redis

Índice

- 0. Objetivo do Sistema
- 1. Sumário Executivo
 - 1.1 Modelo de Operação (Custodial + Treasury Payout)
 - 1.2 Product Contract
 - 1.3 Decisões Finais
 - 1.4 Confirmações por Rede
- 2. Arquitetura do Sistema
 - 2.1 Visão Geral dos Componentes
 - 2.2 Interface dos Executors
 - 2.3 Máquina de Estados
 - 2.4 Custódia e Derivação de Chaves
 - 2.5 Dados Mínimos (Tabelas)
 - 2.6 Watcher / Indexer
- 3. Policy Table v1
 - 3.1 Lightning Network (Futuro)
 - 3.2 Bitcoin On-Chain
 - 3.3 Liquid Network
 - 3.4 Treasury Wallet
- 4. Thresholds em BRL
 - 4.1 Faixas de Valor
 - 4.2 SLOs por Rede
 - 4.3 Limites Operacionais
- 5. Especificação de Validações
 - 5.1 Códigos de Erro
 - 5.2 Validações por Rede
 - 5.3 Regras de Negócio
- 6. Fluxos de Execução
 - 6.1 Fluxos Integrados (Wallet + UTXO Index)
 - 6.2 Bitcoin Flow
 - 6.3 Lightning Flow (Futuro)
 - 6.4 Notas Técnicas de Implementação (MVP)
- 7. Resiliência
 - 7.1 Circuit Breaker
 - 7.2 Retry Policy
- 8. Auditoria e Observabilidade
 - 8.1 Schema de Auditoria
 - 8.2 Métricas e Alertas
- 9. Runbook Operacional
 - 9.1 RBF/CPFP
 - 9.2 DLQ
 - 9.3 Consolidação de UTXOs
- 10. Checklist de Production Readiness
- 11. Configuração de Referência

0. Objetivo do Sistema

Resolver o problema de controle de saldo por endereço (on-chain) e permitir integração segura e auditável entre depósitos on-chain, índice de UTXOs e payouts multi-rede.

1. Depósitos/recebimentos rastreados pela blockchain (principalmente USDT na Liquid)
2. Saldo por usuário derivado on-chain (UTXO por endereço), com reservas internas de UTXO
3. Saques/payouts (BTC/Liquid) com idempotência, retry e DLQ
4. Integração nativa com o fluxo do app DominiPay (PIX/Depix)

Nota realista: o saldo é derivado da própria blockchain (UTXOs por endereço) e das reservas internas de UTXO (lock/hold). Não usamos ledger de crédito/débito no MVP; se necessário para contabilidade, pode ser adicionado sem alterar a fonte de verdade on-chain.

1. Sumário Executivo

DomniWallet Wallet + Payout Service é um sistema **custodial por API** que controla saldo por usuário a partir do índice on-chain de UTXOs (por endereço) e depósitos on-chain (principalmente USDT na Liquid), além de executar payouts multi-rede (BTC e L-BTC). Lightning fica fora do MVP atual. DominiPay integra PIX/Depix e consulta saldo/saques via Wallet API.

1.1 Modelo de Operação (Custodial + Treasury Payout)

Modelo Custodial: BTC usa derivação da master seed (BIP86/P2TR). Liquid gera endereços diretamente no `elements` (wallet do node). O cliente não controla a seed; a DomniWallet custodia fundos e opera um índice on-chain de UTXOs + reservas internas para saques.

Aspecto	Descrição
Custódia de Clientes	SIM — chave do serviço sob custódia do Wallet Service (vault/HSM); seed de recuperação do usuário é entregue uma única vez e não armazenada
Tesouraria	Carteiras operadas pela empresa para payouts
Fluxo	Depósitos confirmados → indexação de UTXOs → saques/payouts
Risco	Empresa assume risco de custódia e liquidez; cliente assume risco de endereço/invoice

Tabela 1.1: Modelo de Operação

Gatilho de Payout (Depix)

Decisão operacional (v1): o saldo spendable é liberado após `depix_conf=2`. No **Modelo A**, o gateway Depix deposita direto no endereço do usuário e o Watcher indexa o UTXO. No **Modelo B** (híbrido), DominiPay enviaria `depix_confirmed` para o Wallet Service, mas este modelo requer ledger interno e fica fora do MVP on-chain. Payouts só são criados com saldo disponível (UTXOs não reservados).

Fluxo atual: PIX recebido → Depix enviado → UTXO confirmado (Modelo A) → `POST /v1/withdrawals` → Wallet Service chama Payout Service.

1.2 Product Contract

Definição clara de como destinos são gerenciados por rede:

Rede	Tipo de Destino	Requisito
BTC	Address Book	Endereço fixo cadastrado por usuário (Bech32m / Taproot preferido)
Liquid	Address Book	Endereço custodial gerado no <code>elements</code> (confidential recomendado)
Tron	Address Book	Endereço TRC20 (USDC) por usuário
Lightning (futuro)	Invoice por Payout	Fora do MVP; manter como extensão futura

Tabela 1.2: Product Contract por Rede

Lightning (futuro): não tem endereço fixo; requer invoice BOLT11 por payout. Fica fora do MVP.

Contrato de Integração DominiPay → Wallet Service (MVP)

#	Regra	Detalhe
1	BTC/Liquid/Tron: <code>to_address</code> fixo	DominiPay envia endereço do address book do cliente. Validação: <code>validateaddress</code>
2	LN (futuro): <code>ln_invoice</code> por payout	Fora do MVP; validações de invoice ficam para versão futura
3	Gatilho: saldo disponível	DominiPay cria <code>withdrawal</code> quando UTXOs confirmados estiverem disponíveis (Modelo A)

Tabela 1.3: Contrato de Integração MVP

1.3 Decisões Finais Aprovadas

Decisão	Resultado	Justificativa
Fallback entre redes	❌ Não automático	Altera expectativa do usuário; fallback via opt-in manual
RBF para BTC	✅ ON por padrão	Resolve stuck tx com baixa complexidade via <code>bumpfee</code>
CPFP para BTC	⚠️ Opcional	Somente se controlamos o output de change
Zero-amount invoices (LN)	❌ Fora do MVP	Aplicável apenas a LN; manter para versão futura
MPP (Multi-Path, LN)	❌ Fora do MVP	Aplicável apenas a LN; manter para versão futura
Webhook Depix	⚠️ Verificar	Confirmar se assinatura implementada

Tabela 1.4: Decisões Finais Aprovadas

1.4 Confirmações por Rede

Rede	Confirmações	Justificativa
Liquid (Depix)	2	Finalidade após 2 conf (~2 min). Documentação Blockstream.
BTC (baixo risco)	1	Valores < 0.01 BTC, liberação rápida
BTC (médio risco)	3	Valores 0.01–0.1 BTC
BTC (alto risco)	6	Valores > 0.1 BTC ou usuário novo

2. Arquitetura do Sistema

2.1 Visão Geral dos Componentes

O sistema completo combina custódia (Wallet Service), observação de blockchain (Watchers) e execução de payouts (Payout Service). O Wallet Service expõe saldo/histórico e cria withdrawals; Watchers indexam UTXOs confirmados; o Payout Service reserva UTXOs e executa payouts BTC/Liquid.

Componente	Responsabilidade
DominiPay Backend	Identidade (uuid), PIX/checkout, decide asset de recebimento/payout e integra Wallet API
Wallet Service (custodial)	Gerencia contas, deriva endereços, expõe saldo/histórico e cria withdrawals
Watchers / Indexers	Observam blockchain/mempool, detectam depósitos e escrevem UTXOs confirmados (chain_utxos)
Wallet DB	Fonte de verdade on-chain: accounts, addresses, chain_utxos, reservas e withdrawals
Payout Service	Orquestra saques, executa payouts por rede e monitora confirmações
Nodes locais	bitcoin, elementsd e trond (RPC + wallet)

Tabela 2.1: Componentes do Sistema

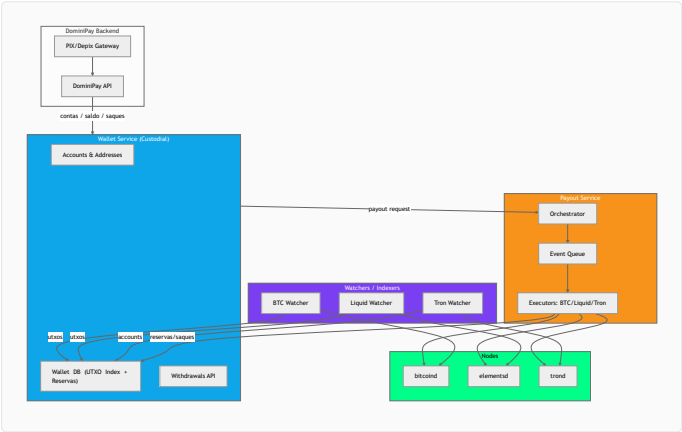


Figura 2.1: Diagrama de Arquitetura do Sistema

2.2 Interface dos Executores

```
type NetworkExecutor interface {
    // Validação
    ValidateAddress(addr string) (bool, error)

    // Execução
    SendPayout(req PayoutRequest) (*PayoutResult, error)

    // Monitoramento
    WatchConfirmations(txid string) ←chan ConfirmationEvent
    GetBalance() (*Balance, error)
}
```

Listagem 2.1: Interface NetworkExecutor

2.3 Máquina de Estados

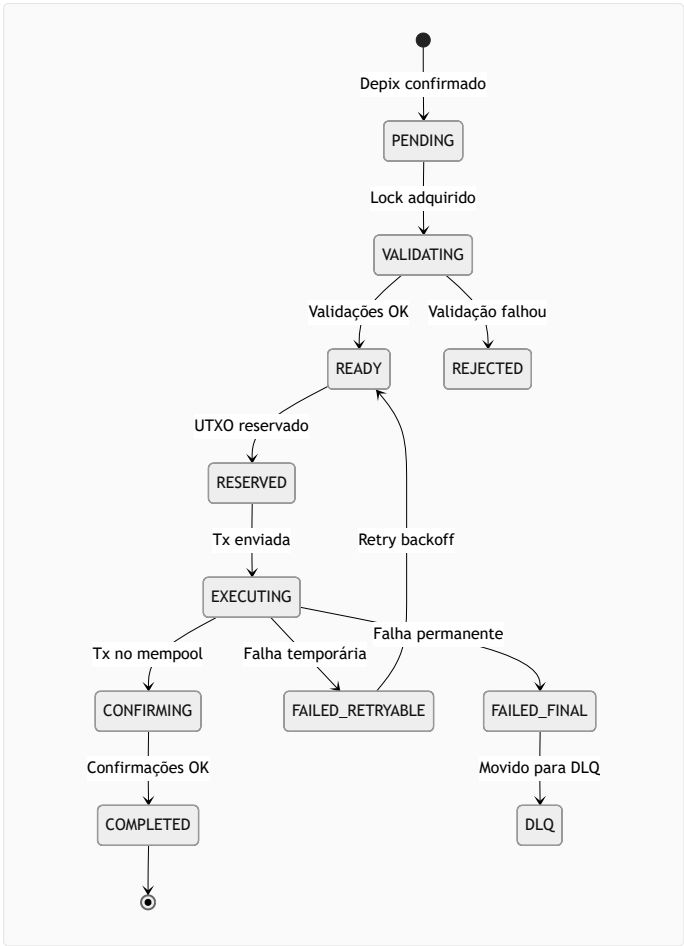


Figura 2.2: Máquina de Estados do Payout

De	Para	Trigger	Ação
PENDING	VALIDATING	Evento de fila	Adquirir lock por <code>payment_id</code>
READY	RESERVED	Validação OK	BTC/Liquid: Coin reservation (marcar UTXOs)
RESERVED	EXECUTING	UTXO reservado	Criar/assinar/broadcast tx
CONFIRMING	COMPLETED	Confirmações atingidas	Liberar lock, persistir proof
FAILED_RETRYABLE	READY	Timer backoff	Incrementar <code>retry_count</code>
FAILED_RETRYABLE	FAILED_FINAL	<code>retry_count</code> $\geq$ 3	Mover para DLQ

Tabela 2.2: Transições de Estado

2.4 Custódia e Derivação de Chaves

O Wallet Service é custodial para operações do dia a dia. Para BTC, usa P2TR com dois caminhos: key-path (chave do serviço) para saques normais e script-path de recuperação do usuário com CSV por tier de risco. Cada usuário recebe um `account_index` determinístico (hardened) para o key-path BIP86. O sistema gera uma seed de recuperação para o usuário, entrega uma única vez e não armazena; apenas o pubkey/xpub é guardado para compor o script de recuperação. Para Liquid, os endereços são gerados diretamente no `elements` (wallet do node), associados ao `account_id` por label.

Item	Decisão
Conta por usuário	<code>accounts(uuid → account_index)</code> determinístico
BTC (custódia)	P2TR key-path (serviço): <code>m/86'/0'/account'/0/i</code>
BTC (recuperação)	Script-path com CSV por tier de risco; pubkey do usuário derivado de seed gerada e entregue uma única vez
Seed do usuário	Gerada pelo sistema, entregue uma única vez e não armazenada; guardar apenas pubkey/xpub
Liquid	Endereços gerados no <code>elements</code> (wallet do node); label por <code>account_id</code>
Tron	BIP44 (coin type 195): <code>m/44'/195'/account'/0/i</code> (custodial)
UUID ≠ PubKey	Não usar uuid como pubkey (mistura identidade com criptografia)

Tabela 2.3: Modelo de Custódia e Derivação

2.4.1 Recuperação (Break-glass) BTC

O caminho de recuperação do usuário usa script-path com CSV por tier de risco. O sistema gera a seed de recuperação e entrega uma única vez; não armazena a seed. Apenas o `user_xpub` (por conta) é guardado para compor o script. A recuperação exige reconstruir o descriptor P2TR (key-path do serviço + script-path CSV com pubkey do usuário) e varrer os UTXOs após o timelock.

- **Padrão recomendado:** `user_xpub` por conta (não pubkey única) para permitir recuperação por endereço e rotação segura.

- **Compatibilidade:** a recuperação deve usar wallet que aceite descriptors P2TR com CSV (ou fluxo de sweep custom).
- **UX de recuperação:** fornecer ao usuário o `user_xpub` , policy de CSV e exemplo de derivação `m/86'/0'/account'/0/1` para reconstrução.

Segurança mínima: seed do serviço armazenada em vault/TPM/HSM (ou encrypted-at-rest + unlock controlado). A seed de recuperação do usuário é entregue uma única vez e não é armazenada (guardar apenas pubkey/xpub). Hot wallets com limites e política de backup/rotação.

2.5 Dados Mínimos (Tabelas)

Entidade	Campos-chave
accounts	uuid (UNIQUE), account_index, user_xpub (BTC), status
addresses	account_id, network, asset, address, derivation_path, is_active
chain_utxos	network, asset, txid, vout, address_id, amount, confirmations, confirmed_at, spent_txid
chain_accounts	network (tron), asset, address_id, balance, last_block, last_txid
utxo_reservations	network, asset, txid, vout, withdrawal_id, reserved_at, status
withdrawals	account_id, network, asset, amount, destination, status
payouts	withdrawal_id, status, txid/payment_hash, error

Tabela 2.4: Entidades Mínimas

Idempotência de depósitos: índice único em `chain_utxos` (`network`, `asset`, `txid`, `vout`) garante registro único por UTXO/asset.

2.6 Watcher / Indexer (Observação de Depósitos)

Como observar endereços na prática:

- **Opção 1 (rápida, MVP):** usar wallet RPC dos nodes; importar descriptors e listar transações. Prós: rápida; Contras: depende de indexing e rescan.
- **Opção 2 (robusta):** indexador próprio + mempool/esplora. Prós: escala melhor; Contras: mais trabalho.

Recomendação: iniciar com Opção 1 + reconciliação periódica. Evoluir para Opção 2 quando throughput justificar.

Robustez do watcher: tratar reorgs, UTXO que some, transações RBF substituídas e confirmações revertidas (confirmed → mempool). Persistir `block_hash` e `height` por UTXO e ter reconciliação periódica para corrigir o estado. Para redes account-based (Tron), persistir saldo + último bloco e reconciliar transferências de token.

3. Policy Table v1

Valores iniciais para implementação, prontos para ajuste baseado em métricas de produção.

Escopo do MVP: apenas BTC e Liquid. Lightning fica fora do MVP e é referência futura.

3.1 Lightning Network (Futuro - fora do MVP)

3.1.1 Parâmetros por Faixa de Valor

Faixa	timeout	cltv_limit	fee_limit	MPP	max_parts	max_shard
< 100k sats	30s	40	5000 ppm, min 5, max 500 sat	OFF	6	50k sat
100k–1M sats	60s	60	3000 ppm, min 20, max 2k sat	ON	10	100k sat
> 1M sats	90s	80	2000 ppm, min 50, max 5k sat	ON	12	200k sat

Tabela 3.1: Parâmetros LN por Faixa

3.1.2 Regras Auxiliares LN

- Validar invoice: expiry , num_satoshis , payment_hash , destination
- Bloquear se now + timeout_seconds estiver a menos de 20% do expiry
- Zero-amount: aceitar somente se payout define valor; enviar via amt_msat
- Persistir payment_preimage + status após SUCCEEDED

3.2 Bitcoin On-Chain

Parâmetro	Valor
Confirmações (baixo risco)	1
Confirmações (médio risco)	3
Confirmações (alto risco)	6
RBF	ON por padrão
CPFP	Opcional (quando controlamos change)
Fee estimation targets	2 / 6 / 12 blocos
Fee fallback	10 sat/vB

Tabela 3.2: Parâmetros BTC

3.2.1 Timelock CSV (Recuperação BTC)

Aplica-se apenas ao script-path de recuperação do usuário. O key-path do serviço não é afetado.

Faixa (BRL)	CSV (blocos)	Tempo aprox.
Pequeno (< R\$ 600)	144	~1 dia
Médio (R\$ 600–6.000)	432	~3 dias
Grande (> R\$ 6.000)	1008	~1 semana

Tabela 3.2a: Timelock CSV por Faixa

3.3 Liquid Network

Parâmetro	Valor
Depix confirmações	2
Payout confirmações	2
Fee fallback	0.1 sat/vB

Tabela 3.3: Parâmetros Liquid

3.4 Tron Network

Parâmetro	Valor
Confirmações (USDC TRC20)	19 (ajustável)
Fee model	Energy/Bandwidth (manter saldo de TRX)

Tabela 3.4: Parâmetros Tron

3.5 Treasury Wallet Limits

Limites para as carteiras da tesouraria operacional (não-custodial de clientes):

Rede	min_balance	target_balance	max_balance
BTC	0.05 BTC	0.2 BTC	0.5 BTC
Liquid (L-BTC)	0.02 L-BTC	0.1 L-BTC	0.3 L-BTC
Lightning	500k sats	2M sats	5M sats

Tabela 3.4: Limites de Treasury Wallet

Importante: Reposição da cold → treasury wallet requer **aprovação humana** via dashboard ou CLI. Nunca automático.

4. Thresholds em BRL

Taxa de conversão de referência: 1 BTC = R\$ 600.000 (ajustar conforme mercado).

4.1 Faixas de Valor

Faixa	BRL	Sats	Classificação
Pequeno	< R\$ 600	< 100k	Baixo risco
Médio	R\$ 600 – R\$ 6.000	100k – 1M	Médio risco
Grande	> R\$ 6.000	> 1M	Alto risco

Tabela 4.1: Faixas de Valor em BRL

4.2 SLOs por Rede

4.2.1 Lightning Network (Futuro)

Métrica	Target	Janela
Taxa de sucesso	≥ 98%	24h rolling
Latência (p50)	≤ 15s	24h rolling
Latência (p99)	≤ 120s	24h rolling
Fee médio pago	≤ 0.3% do valor	Mensal

Tabela 4.2: SLOs Lightning

4.2.2 Bitcoin On-Chain

Métrica	Target	Janela
Taxa de broadcast	≥ 99.5%	24h
Tempo até 1 conf (p50)	≤ 15 min	24h
Tempo até 1 conf (p99)	≤ 60 min	24h
Tx stuck (> 24h)	≤ 0.1%	Semanal

Tabela 4.3: SLOs Bitcoin

4.3 Limites Operacionais

4.3.1 Por Payout

Rede	Mínimo	Máximo
Lightning	R\$ 1 (~170 sat)	R\$ 60.000 (~10M sat)
BTC	R\$ 60 (~10k sat)	R\$ 300.000 (~50M sat)
Liquid	R\$ 6 (~1k sat)	R\$ 300.000 (~50M sat)

Tabela 4.4: Limites por Payout

4.3.2 Por Usuário (Diário)

Tier	Limite Diário
Básico	R\$ 10.000
Verificado	R\$ 50.000
Premium	R\$ 200.000

Tabela 4.5: Limites por Usuário

5. Especificação de Validações

5.1 Códigos de Erro

5.1.1 Validação (4xx)

Código	HTTP	Descrição	Retryable
PAYOUT_INVALID_ADDRESS	400	Endereço inválido	✗
PAYOUT_MIXED_CASE_BECH32	400	Bech32 com mixed-case	✗
PAYOUT_INVALID_INVOICE	400	Invoice LN malformada	✗
PAYOUT_INVOICE_EXPIRED	400	Invoice já expirada	✗
PAYOUT_INVOICE_NEAR_EXPIRY	400	Invoice < 20% tempo restante	✗
PAYOUT_AMOUNT_MISMATCH	400	Valor difere do esperado	✗
PAYOUT_DUPLICATE	409	payment_id já processado	✗
PAYOUT_LIMIT_EXCEEDED	429	Limite diário excedido	✗

Tabela 5.1: Códigos de Erro de Validação

5.1.2 Execução (5xx)

Código	HTTP	Descrição	Retryable
PAYOUT_INSUFFICIENT_BALANCE	503	Saldo insuficiente	✓
PAYOUT_NETWORK_UNAVAILABLE	503	Rede indisponível	✓
PAYOUT_CIRCUIT_OPEN	503	Circuit breaker aberto	✓
PAYOUT_LN_NO_ROUTE	502	Sem rota LN disponível	✓
PAYOUT_LN_FEE_TOO_HIGH	502	Fee LN excede limite	✓
PAYOUT_TX_REJECTED	502	Tx rejeitada pelo node	✗

Tabela 5.2: Códigos de Erro de Execução

5.2 Validações por Rede

5.2.1 BTC Address

```
func ValidateBTCAddress(addr string) error {
    // 1. Verificar mixed-case (BIP-173)
    if hasMixedCase(addr) && strings.HasPrefix(strings.ToLower(addr), "bc1") {
        return ErrMixedCaseBech32
    }
    // 2. Chamar bitcoind validateaddress
    resp, _ := rpc.Call("validateaddress", addr)
    if !resp.IsValid {
        return ErrInvalidAddress
    }
    return nil
}
```

Listagem 5.1: Validação de Endereço BTC

5.2.2 Lightning Invoice (Futuro)

```
func ValidateLNInvoice(invoice string, expectedAmount int64) error {
    decoded, err := lnd.DecodePayReq(invoice)
    if err != nil {
        return ErrInvalidInvoice
    }

    // Expiração
    expiresAt := decoded.Timestamp + decoded.Expiry
    if time.Now().Unix() > expiresAt {
        return ErrInvoiceExpired
    }

    // 80% do expiry
    remaining := expiresAt - time.Now().Unix()
    if remaining < int64(decoded.Expiry)*0.2 {
        return ErrInvoiceNearExpiry
    }

    // Amount
    if decoded.NumSatoshis == 0 {
        // Zero-amount: aceitar se expectedAmount > 0
    } else if decoded.NumSatoshis != expectedAmount {
        return ErrAmountMismatch
    }

    return nil
}
```

Listagem 5.2: Validação de Invoice LN

5.3 Regras de Negócio

```
-- Constraint de unicidade
CREATE UNIQUE INDEX idx_payout_idempotency
ON payouts (payment_id, network);

-- Lock + Check
BEGIN;
SELECT * FROM payouts
WHERE payment_id = $1 AND network = $2
FOR UPDATE NOWAIT;

INSERT INTO payouts (payment_id, network, status, ...)
VALUES ($1, $2, 'PENDING', ...)
```

```
ON CONFLICT DO NOTHING
RETURNING id;
COMMIT;
```

Listagem 5.3: Idempotência via SQL

6. Fluxos de Execução

6.1 Fluxos Integrados (Wallet + UTXO Index)

6.1.1 Onboarding (conta + endereço)

- 1. DominiPay cria usuário e chama `POST /v1/accounts` com `uuid`
- 2. DominiPay solicita endereços por rede/asset (ex.: Liquid USDT, Liquid L-BTC)
- 3. Wallet Service deriva e retorna o endereço, gravando em `addresses`

6.1.2 Depósito normal (USDT Liquid)

- 1. Cliente deposita USDT na Liquid no endereço do usuário
- 2. Watcher detecta UTXO (`txid:vout` , `asset_id`, `amount`, `address`)
- 3. Após `confirmations >= N` , grava em `chain_utxos`
- 4. Saldo é derivado de `chain_utxos` (menos UTXOs reservados)
- 5. DominiPay consulta saldo via `GET /v1/accounts/{uuid}/balances`

6.1.2b Depósito normal (USDC Tron)

- 1. Cliente deposita USDC (TRC20) no endereço Tron do usuário
- 2. Watcher detecta transferência (`txid`, `contract`, `amount`, `address`)
- 3. Após `confirmations >= N` , grava em `chain_accounts`
- 4. Saldo é derivado de `chain_accounts` (menos reservas)
- 5. DominiPay consulta saldo via `GET /v1/accounts/{uuid}/balances`

6.1.3 Fluxo PIX → Depix → Crédito

Evolução futura (ledger): se for necessário saldo lógico (Modelo B), definir eventos geradores (`depix_confirmed` , `swap_fill` , `estorno`) e um reconciliador que concilia ledger vs on-chain sem quebrar a fonte de verdade.

Dois modelos possíveis, dependentes do gateway Depix:

Modelo A (suportado no MVP): Depix deposita direto no endereço do usuário.
PIX confirmado → Depix envia para endereço USDT do usuário → Watcher indexa UTXO automaticamente.

Modelo B (híbrido): Depix deposita na tesouraria.
PIX confirmado → DominiPay emite evento `depix_confirmed` → este modelo exige ledger interno e fica fora do escopo do MVP on-chain.

6.1.4 Payout (saque) BTC/Liquid/Tron

- 1. DominiPay chama `POST /v1/withdrawals` com rede, asset, amount e destination
- 2. Wallet Service valida saldo on-chain e destino; reserva UTXOs (`lockunspent`) ou saldo lógico (Tron)
- 3. Wallet Service chama Payout Service com idempotência
- 4. Payout Service executa (BTC/Liquid/Tron) e monitora confirmações
- 5. Ao concluir: marcar UTXOs como spent e status `COMPLETED`
- 6. Se falhar: liberar reservas/locks

6.1.5 APIs e Eventos

Fluxo	Endpoint	Descrição
Wallet Service	<code>POST /v1/accounts</code>	Cria conta para <code>uuid</code>
Wallet Service	<code>POST /v1/accounts/{uuid}/addresses</code>	Gera endereço por rede/asset
Wallet Service	<code>GET /v1/accounts/{uuid}/balances</code>	Consulta saldo por asset
Wallet Service	<code>GET /v1/accounts/{uuid}/transactions</code>	Extrato e histórico
Wallet Service	<code>POST /v1/withdrawals</code>	Cria saque e reserva UTXOs
Wallet Service	<code>GET /v1/withdrawals/{id}</code>	Status do saque
Eventos DominiPay	<code>POST /v1/events/depix_confirmed</code>	Modelo B; idempotência por <code>payment_id</code>
Wallet → Payout Service	<code>POST /v1/payouts</code>	Executa payout (idempotente)

Tabela 6.1: APIs e Eventos do Fluxo Integrado

6.2 Bitcoin Flow

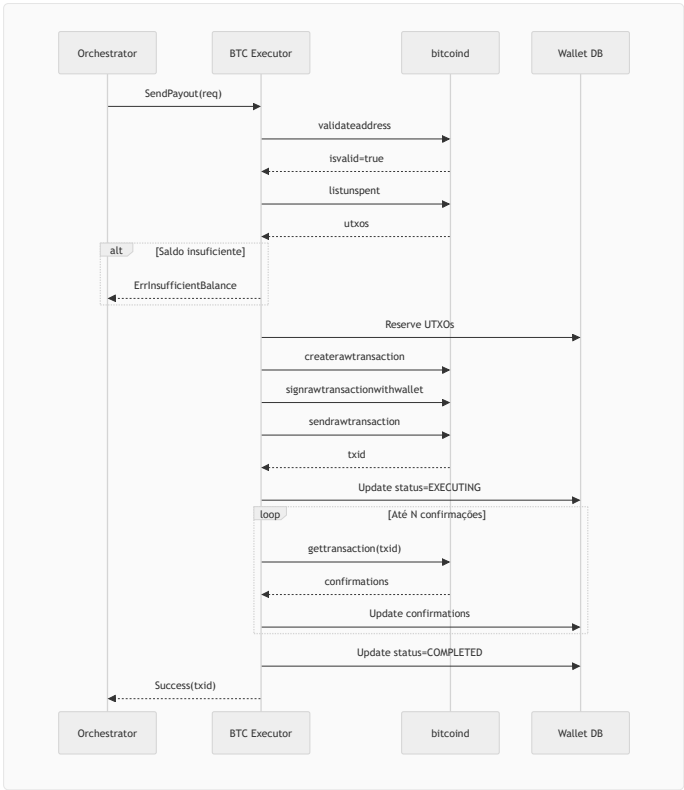


Figura 6.2: Fluxo de Payout BTC

6.3 Lightning Flow (Futuro - fora do MVP)

Fora do MVP: esta seção é referência futura e não faz parte do escopo atual.

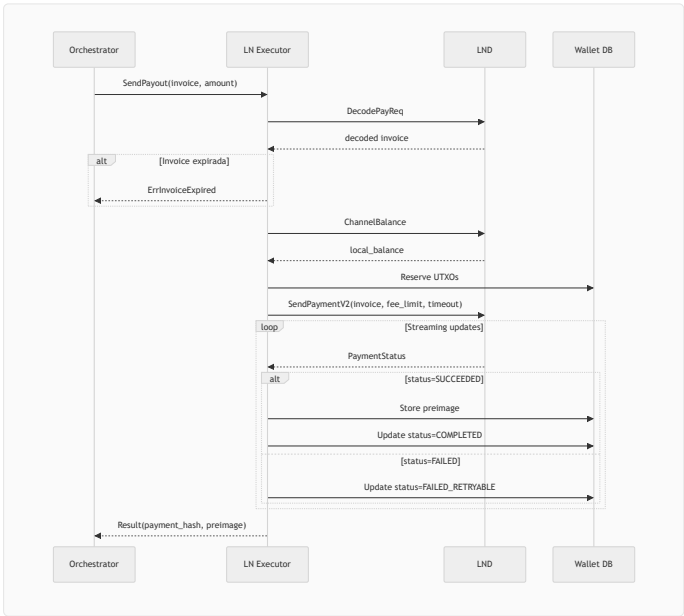


Figura 6.3: Fluxo de Payout Lightning

6.4 Notas Técnicas de Implementação (MVP)

Anti-fraude: aplicar limite por withdrawal e cooldown para mudança de destination/ address book (por tier). Alterações recentes devem bloquear saques até o cooldown expirar.

Rede	Requisito Técnico	Implementação
BTC/Liquid	PSBT Funding + UTXO Lock	Usar <code>walletcreatefundedpsbt</code> ou <code>fundrawtransaction</code> para funding automático + fee. Reservar UTXOs específicos via <code>lockunspent</code> antes do broadcast. RBF já decidido ON.
BTC/Liquid	Unlock em falha	Se tx falhar antes de broadcast, chamar <code>lockunspent(true, utxos)</code> para liberar.

Tabela 6.2: Requisitos Técnicos MVP

Sem UTXO lock: "saldo tem, mas não dá pra gastar" — tx fica presa aguardando inputs já usados.

7. Resiliência

7.1 Circuit Breaker

Circuit breakers são **independentes por rede**. BTC aberto NÃO bloqueia Liquid.

```
type CircuitBreaker struct {
    State      string    // CLOSED, OPEN, HALF_OPEN
    FailureCount int       //
    FailureThreshold int    // default: 5
    OpenDuration time.Duration // default: 60s
    LastFailure time.Time
    Queue      []PayoutRequest // Fila preservada
    MaxQueueSize int        // default: 100 (backlog limit)
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    if cb.State == "OPEN" {
        if time.Since(cb.LastFailure) > cb.OpenDuration {
            cb.State = "HALF_OPEN"
        } else {
            // Shed load se backlog exceder limite
            if len(cb.Queue) >= cb.MaxQueueSize {
                return ErrBackLogFull // Retryable, não enfileira
            }
            return ErrCircuitOpen // Enqueue
        }
    }

    err := fn()
    if err != nil {
        cb.FailureCount++
        cb.LastFailure = time.Now()
        if cb.FailureCount >= cb.FailureThreshold {
            cb.State = "OPEN"
        }
        return err
    }

    cb.FailureCount = 0
    cb.State = "CLOSED"
    return nil
}
```

Listagem 71: Circuit Breaker com Backlog Limit

Proteção Financeira: Quando circuit abre, payouts são **enfileirados até o limite**. Se `len(queue) >= MaxQueueSize`, novos payouts são rejeitados com `ErrBackLogFull` (retryable). Isso evita "fila de dívida" em burst de Depix.

Parâmetro	Valor Padrão	Descrição
MaxQueueSize	100 por rede	Limite de backlog; acima disso, shed load
FailureThreshold	5	Falhas consecutivas para abrir circuit
OpenDuration	60s	Tempo em OPEN antes de tentar HALF_OPEN

Tabela 70: Parâmetros do Circuit Breaker

7.2 Retry Policy

7.2.1 Lightning Network (Futuro)

Tentativa	Delay	Ajuste
1	imediato	Parâmetros normais
2	5s	<code>max_hops</code> reduzido
3	15s	<code>fee_limit</code> +20%
4	—	Marcar FAILED_RETRYABLE

Tabela 71: Retry Policy LN

7.2.2 Condições de Abort

Condição	Ação
Invoice > 80% expiry consumido	Abortar, solicitar nova invoice
<code>insufficient_balance</code>	Abortar imediatamente, alertar ops
<code>payment_hash_reuse</code>	Não retry, marcar duplicate

Tabela 72: Condições de Abort

8. Auditoria e Observabilidade

8.1 Schema de Auditoria

```
CREATE TABLE payout_audit (  
  id BIGSERIAL PRIMARY KEY,  
  payout_id BIGINT REFERENCES payouts(id),  
  payment_id VARCHAR(64) NOT NULL,  
  network VARCHAR(10) NOT NULL,  
  
  -- Prova  
  txid VARCHAR(64),      -- BTC/Liquid  
  payment_hash VARCHAR(64), -- LN  
  preimage_hash VARCHAR(64), -- SHA256(preimage)  
  
  -- Hash chain  
  prev_hash VARCHAR(64),  
  record_hash VARCHAR(64), -- SHA256(prev || payment_id || txid)  
  
  -- Metadata  
  amount_sats BIGINT,  
  fee_paid_sats BIGINT,  
  confirmations INT,  
  status VARCHAR(20),  
  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- Trigger para impedir UPDATE (WORM)  
CREATE TRIGGER audit_immutable  
BEFORE UPDATE ON payout_audit  
FOR EACH ROW EXECUTE FUNCTION prevent_audit_update();
```

Listagem 8.1: Schema de Auditoria Append-Only

8.2 Métricas Prometheus

```
# Counters  
domini_payout_total{network, status}  
domini_payout_retries_total{network}  
  
# Histograms  
domini_payout_duration_seconds{network}  
domini_payout_confirmations_wait_seconds{network}  
domini_ln_fee_paid_sats{bucket}  
  
# Gauges  
domini_hot_wallet_balance_sats{network}  
domini_utxo_count{network}  
domini_circuit_breaker_state{network} # 0=closed, 1=open, 2=half_open
```

Listagem 8.2: Métricas Prometheus

8.3 Alertas Recomendados

Alerta	Condição	Severidade
HotWalletLow	balance < min_balance por 5 min	Warning
HotWalletCritical	balance < min_balance * 0.5	Critical
CircuitBreakerOpen	state == OPEN por 2 min	Critical
HighFailureRate	failed / total > 0.1 em 5 min	Warning
UTXOFragmentation	utxo_count > 100	Warning
PayoutLatencyHigh	p99 > 120s (LN) ou > 30min (BTC)	Warning

Tabela 8.1: Alertas Recomendados

9. Runbook Operacional

9.1 RBF - Replace-By-Fee

Para transações BTC stuck no mempool por mais de 1 hora:

```
# 1. Verificar status da tx
bitcoin-cli gettransaction <txid>

# 2. Calcular novo fee (target: próximo bloco)
NEW_FEE=$(bitcoin-cli estimatesmartfee 2 | jq '.feerate')

# 3. Bump fee
bitcoin-cli bumpfee <txid> '{"fee_rate": '$NEW_FEE'}'

# 4. Verificar nova tx
bitcoin-cli gettransaction <new_txid>
```

Listagem 9.1: Procedimento RBF

Pré-requisito: Tx original deve ter sido criada com `replaceable=true` .

9.2 DLQ - Dead Letter Queue

9.2.1 O que vai para DLQ

- Payouts com 3+ falhas
- Erros não-retryable
- Timeouts esgotados

9.2.2 Processamento

```
# Listar itens
curl https://api.domini.pay/v1/admin/dlq?limit=50

# Retry forçado
curl -X POST https://api.domini.pay/v1/admin/dlq/<id>/retry

# Resolução manual
curl -X POST https://api.domini.pay/v1/admin/dlq/<id>/resolve \
-d '{"resolution": "manual_refund", "notes": "PIX"}'
```

Listagem 9.2: Comandos DLQ

9.3 Consolidação de UTXOs

Trigger	Ação
UTXOs > 50	Consolidar no domingo 02:00 UTC
Fees < 5 sat/vB	Consolidar imediatamente

Tabela 9.1: Triggers de Consolidação

```
# Contar UTXOs
bitcoin-cli listunspent | jq 'length'

# Consolidar
bitcoin-cli sendtoaddress "<hot_wallet>" \
$(bitcoin-cli getbalance) \
"" "" true true null "unset" null $FEE_RATE
```

Listagem 9.3: Consolidação Manual

10. Checklist de Production Readiness

10.1 Funcionalidades Core

- ☐ BTC Executor: send, watch, confirm
- ☐ Liquid Executor: send, watch, confirm
- ☐ LN Executor: decode, send, track
- ☐ State machine completa
- ☐ Idempotência por payment_id + network
- ☐ Lock distribuído (DB ou Redis)
- ☐ Reserva de saldo pré-envio

10.2 Resiliência

- ☐ Circuit breaker por rede (com fila)
- ☐ Retry policy com backoff
- ☐ DLQ para falhas finais
- ☐ Fallback de fee estimation
- ☐ Timeout configurável por rede

10.3 Segurança

- ☐ Preimage LN criptografado em repouso
- ☐ Hot wallet limits enforced
- ☐ Aprovação humana para reposição
- ☐ Auditoria append-only
- ☐ Hash chain para integridade

10.4 Operacional

- ☐ Consolidação de UTXOs (cron)
- ☐ Métricas Prometheus
- ☐ Alertas configurados
- ☐ Runbook de DLQ
- ☐ Dashboard de monitoramento

10.5 Testes

- ☐ Unit tests: 80%+ cobertura
- ☐ Integration tests: cada executor
- ☐ Chaos tests: kill mid-payout
- ☐ Replay test: idempotência validada
- ☐ Load test: throughput esperado

10.6 SLOs Propostos

Métrica	Target	Medição
Disponibilidade	99.9%	Uptime do Payout Service
Latência LN (p99)	< 120s	Da requisição ao COMPLETED
Latência BTC (p99)	< 60 min	Da requisição à 1ª confirmação
Taxa de sucesso	> 98%	Payouts COMPLETED / Total
Falhas não recuperadas	< 0.1%	Payouts em DLQ / Total

Tabela 10.1: SLOs Propostos

11. Configuração de Referência

```
ln:
  timeout_seconds:
    small: 30 # < 100k sats
    medium: 60 # 100k-1M sats
    large: 90 # > 1M sats
  cltv_limit:
    small: 40
    medium: 60
    large: 80
  fee_limit_ppm:
    small: 5000 # 0.5%
    medium: 3000 # 0.3%
    large: 2000 # 0.2%
  fee_limit_min_sat:
    small: 5
    medium: 20
    large: 50
  fee_limit_max_sat:
    small: 500
    medium: 2000
    large: 5000
  mpp:
    enable_threshold_sat: 100000
    max_parts:
      small: 6
      medium: 10
      large: 12
    max_shard_size_sat:
      small: 50000
      medium: 100000
      large: 200000
  retry:
    max_attempts: 3
    backoff_seconds: [5, 15, 45]
    expiry_threshold_pct: 0.8

btc:
  confs_final:
    low: 1
    medium: 3
    high: 6
  rbf: true
  fee_targets_blocks: [2, 6, 12]
  fee_fallback_sat_vb: 10

liquid:
  depix_confs: 2
  payout_confs: 2
  fee_fallback_sat_vb: 0.1

hot_wallet:
  btc:
    min_balance_sat: 5000000 # 0.05 BTC
    target_balance_sat: 20000000 # 0.2 BTC
    max_balance_sat: 50000000 # 0.5 BTC
  liquid:
    min_balance_sat: 2000000
    target_balance_sat: 10000000
    max_balance_sat: 30000000
  lightning:
    min_balance_sat: 500000
    target_balance_sat: 2000000
    max_balance_sat: 5000000

circuit_breaker:
  failure_threshold: 5
  open_duration_seconds: 60

consolidation:
  utxo_threshold: 50
  schedule: "0 2 * * 0" # Domingo 02:00
```

Listagem 11.1: Configuração YAML Completa

Documento aprovado para implementação. Revisar semanalmente a taxa de conversão BRL e mensalmente as métricas de SLO para ajustes na Policy Table.